

✓ Welcome to Colab!

```
!pip install -q kaggle
```

```
import os
os.environ["KAGGLE_API_TOKEN"] = "KGAT_beaf6fd5ca2ac5baa0a8660ea821a7b1"
```

```
!kaggle datasets list
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to ref

ref	title
rauffauzanrambe/fifa-world-cup-2026-player-performance-dataset	FIFA World Cup 2026 Player Performance Dataset
ashyou09/global-budget-allocation-dataset-19362026	Global Budget Allocation Dataset (1936-2026) 🌐
abbas829/ecommerce-sales-dataset	Ecommerce sales dataset
ashyou09/apple-products-pricing-dataset-2020-2026	Apple Products Pricing Dataset (2020-2026) 🍏
mann14/2026-fifa-world-cup	2026 FIFA World Cup
uditjain13/neurogolf-kevali-7232-nets	NeuroGolf 2026 - Kevali 7268.48 Nets
udaykumar025/anime-dataset-top-10k-normalized	Anime dataset (Top 10K Normalized)
rewantbhriguvanshi/customer-shopping-trends-indian	Customer Shopping Trends Indian
uditjain13/social-media-screen-time-and-mental-health-2026	Social Media, Screen Time & Mental Health 2026
mirzayasirabdullah07/netflix-dataset	Netflix Dataset
sunil123kumar/social-media-impact-on-mental-health	Social_Media_Impact_on_Mental Health
zynicide/wine-reviews	Wine Reviews
arjunmehta1992/chocolate-sales-in-20222023	Chocolate Sales in 2022-2023
datasnaek/youtube-new	Trending YouTube Video Statistics
uditjain13/nvidia-gpu-sales-synthetic-2026	Nvidia GPU Sales Data 2026
datasnaek/chess	Chess Game Dataset (Lichess)
nolanbconaway/pitchfork-data	18,393 Pitchfork Reviews
rtatman/188-million-us-wildfires	1.88 Million US Wildfires
nasa/kepler-exoplanet-search-results	Kepler Exoplanet Search Results
residentmario/ramen-ratings	Ramen Ratings

```
!kaggle datasets download -d jishnuparayilshibu/a-curated-list-of-image-deblurring-datasets -p /content/data
```

Warning: Looks like you're using an outdated `kaggle` version (installed: 2.0.2), please consider upgrading to
Dataset URL: <https://www.kaggle.com/datasets/jishnuparayilshibu/a-curated-list-of-image-deblurring-datasets>
License(s): CC0-1.0
Downloading a-curated-list-of-image-deblurring-datasets.zip to /content/data
100% 17.1G/17.1G [03:25<00:00, 89.3MB/s]

```
import zipfile
import os

zip_path = "/content/data/a-curated-list-of-image-deblurring-datasets.zip"
extract_path = "/content/data"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall(extract_path)

print("Dataset extracted successfully.")
```

Dataset extracted successfully.

```
import os
base_dir = "/content/data/DBlur"
print(os.listdir(base_dir))
```

```
['RealBlur_R', 'CelebA', 'Gopro', 'HIDE', 'Wider-Face', 'Helen', 'TextOCR', 'RealBlur_J']
```

```
def count_image_files(directory):
    count = 0
    image_extensions = ('.jpg', '.jpeg', '.png', '.gif', '.bmp', '.tiff')
    for root, _, files in os.walk(directory):
        for file in files:
            if file.lower().endswith(image_extensions):
                count += 1
    return count
```

```
print("RealBlur_R images:", count_image_files(os.path.join(base_dir, "RealBlur_R")))
print("RealBlur_J images:", count_image_files(os.path.join(base_dir, "RealBlur_J")))

print("CelebA images:", count_image_files(os.path.join(base_dir, "CelebA")))
print("HIDE images:", count_image_files(os.path.join(base_dir, "HIDE")))

print("Gopro images:", count_image_files(os.path.join(base_dir, "Gopro")))
print("Wider-Face images:", count_image_files(os.path.join(base_dir, "Wider-Face")))

print("Helen images:", count_image_files(os.path.join(base_dir, "Helen")))
print("TextOCR images:", count_image_files(os.path.join(base_dir, "TextOCR")))
```

```
RealBlur_R images: 1960
RealBlur_J images: 1960
CelebA images: 405198
HIDE images: 4050
Gopro images: 6428
Wider-Face images: 11328
Helen images: 4660
TextOCR images: 12000
```

```
import tensorflow as tf
from tensorflow.keras.preprocessing.image import ImageDataGenerator
IMG_SIZE = 256 # Changed from 150 to 256 for consistent U-Net dimensions
BATCH_SIZE = 32
datagen = ImageDataGenerator(
    rescale=1./255,
    validation_split=0.2
)
train_data = datagen.flow_from_directory(
    base_dir,
    target_size=(IMG_SIZE, IMG_SIZE),
    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="training"
)
val_data = datagen.flow_from_directory(
```

```
base_dir,
target_size=(IMG_SIZE, IMG_SIZE),

    batch_size=BATCH_SIZE,
    class_mode="binary",
    subset="validation"
)
```

```
Found 358069 images belonging to 8 classes.
Found 89515 images belonging to 8 classes.
```

```
print(train_data.class_indices)
```

```
{'CelebA': 0, 'Gopro': 1, 'HIDE': 2, 'Helen': 3, 'RealBlur_J': 4, 'RealBlur_R': 5, 'TextOCR': 6, 'Wider-Face':
```

✓ Build the Generator Model

The Generator's role in image deblurring is to take a blurry image as input and produce a deblurred image. A common architecture for this is a U-Net, which is good at capturing both high-level semantic information and fine-grained details.

This U-Net-like generator will consist of:

- **Encoder Path:** Downsampling layers to extract features at different scales.
- **Decoder Path:** Upsampling layers to reconstruct the image from the learned features.
- **Skip Connections:** To pass features from the encoder directly to the decoder, helping preserve details.

```
from tensorflow.keras import layers, Model

def downsample(filters, size, apply_batchnorm=True):
    initializer = tf.random_normal_initializer(0., 0.02)
    result = tf.keras.Sequential()
    result.add(layers.Conv2D(filters, size, strides=2, padding='same',
                             kernel_initializer=initializer, use_bias=False))

    if apply_batchnorm:
        result.add(layers.BatchNormalization())
```

```
result.add(layers.LeakyReLU())
return result

def upsample(filters, size, apply_dropout=False):
    initializer = tf.random_normal_initializer(0., 0.02)
    result = tf.keras.Sequential()
    result.add(layers.Conv2DTranspose(filters, size, strides=2,
                                      padding='same',
                                      kernel_initializer=initializer,
                                      use_bias=False))

    result.add(layers.BatchNormalization())
    if apply_dropout:
        result.add(layers.Dropout(0.5))
    result.add(layers.ReLU())
    return result

def Generator():
    inputs = layers.Input(shape=[IMG_SIZE, IMG_SIZE, 3])

    down_stack = [
        downsample(64, 4, apply_batchnorm=False), # (bs, 75, 75, 64)
        downsample(128, 4), # (bs, 38, 38, 128)
        downsample(256, 4), # (bs, 19, 19, 256)
        downsample(512, 4), # (bs, 10, 10, 512)
        downsample(512, 4), # (bs, 5, 5, 512)
        downsample(512, 4), # (bs, 3, 3, 512)
        downsample(512, 4), # (bs, 2, 2, 512)
    ]

    up_stack = [
        upsample(512, 4, apply_dropout=True), # (bs, 3, 3, 512)
        upsample(512, 4, apply_dropout=True), # (bs, 5, 5, 512)
        upsample(512, 4, apply_dropout=True), # (bs, 10, 10, 512)
        upsample(256, 4), # (bs, 19, 19, 256)
        upsample(128, 4), # (bs, 38, 38, 128)
        upsample(64, 4), # (bs, 75, 75, 64)
    ]

    initializer = tf.random_normal_initializer(0., 0.02)
```

```
last = layers.Conv2DTranspose(3, 4, strides=2,
                              padding='same',
                              kernel_initializer=initializer,
                              activation='tanh') # (bs, 150, 150, 3)

x = inputs

# Downsampling through the model
skips = []
for down in down_stack:
    x = down(x)
    skips.append(x)

skips = reversed(skips[:-1])

# Upsampling and establishing the skip connections
for up, skip in zip(up_stack, skips):
    x = up(x)
    x = layers.Concatenate()([x, skip])

x = last(x)

return Model(inputs=inputs, outputs=x)

generator = Generator()
generator.summary()
```


Model: "functional_13"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 256, 256, 3)	0	-
sequential (Sequential)	(None, 128, 128, 64)	3,072	input_layer[0][0]
sequential_1 (Sequential)	(None, 64, 64, 128)	131,584	sequential[0][0]
sequential_2 (Sequential)	(None, 32, 32, 256)	525,312	sequential_1[0][...]
sequential_3 (Sequential)	(None, 16, 16, 512)	2,099,200	sequential_2[0][...]
sequential_4 (Sequential)	(None, 8, 8, 512)	4,196,352	sequential_3[0][...]
sequential_5 (Sequential)	(None, 4, 4, 512)	4,196,352	sequential_4[0][...]
sequential_6 (Sequential)	(None, 2, 2, 512)	4,196,352	sequential_5[0][...]
sequential_7 (Sequential)	(None, 4, 4, 512)	4,196,352	sequential_6[0][...]
concatenate (Concatenate)	(None, 4, 4, 1024)	0	sequential_7[0][...] sequential_5[0][...]

Build the Discriminator Model

The Discriminator's job is to distinguish between 'real' deblurred images (from the dataset) and 'fake' deblurred images (generated by our Generator). For image-to-image translation, a PatchGAN discriminator is often used, which classifies N patches of the image as real or fake, rather than the whole image.

```
def Discriminator():
    initializer = tf.random_normal_initializer(0., 0.02)

    inp = layers.Input(shape=[IMG_SIZE, IMG_SIZE, 3], name='input_image')
    tar = layers.Input(shape=[IMG_SIZE, IMG_SIZE, 3], name='target_image')

    x = layers.concatenate([inp, tar]) # (bs, 150, 150, channels*2)
```

```
down1 = downsample(64, 4, False)(x) # (bs, 75, 75, 64)
down2 = downsample(128, 4)(down1) # (bs, 38, 38, 128)
down3 = downsample(256, 4)(down2) # (bs, 19, 19, 256)

zero_pad1 = layers.ZeroPadding2D()(down3) # (bs, 21, 21, 256)
conv = layers.Conv2D(512, 4, strides=1,
                    kernel_initializer=initializer,
                    use_bias=False)(zero_pad1) # (bs, 18, 18, 512)

batchnorm1 = layers.BatchNormalization()(conv)

leaky_relu = layers.LeakyReLU()(batchnorm1)

zero_pad2 = layers.ZeroPadding2D()(leaky_relu) # (bs, 20, 20, 512)

last = layers.Conv2D(1, 4, strides=1,
                    kernel_initializer=initializer)(zero_pad2) # (bs, 17, 17, 1)

return Model(inputs=[inp, tar], outputs=last)

discriminator = Discriminator()
discriminator.summary()
```

Model: "functional_17"

Layer (type)	Output Shape	Param #	Connected to
input_image (InputLayer)	(None, 256, 256, 3)	0	-
target_image (InputLayer)	(None, 256, 256, 3)	0	-
concatenate_6 (Concatenate)	(None, 256, 256, 6)	0	input_image[0][0... target_image[0][...
sequential_13 (Sequential)	(None, 128, 128, 64)	6,144	concatenate_6[0]...
sequential_14 (Sequential)	(None, 64, 64, 128)	131,584	sequential_13[0]...
sequential_15 (Sequential)	(None, 32, 32, 256)	525,312	sequential_14[0]...
zero_padding2d (ZeroPadding2D)	(None, 34, 34, 256)	0	sequential_15[0]...
conv2d_10 (Conv2D)	(None, 31, 31, 512)	2,097,152	zero_padding2d[0]...
batch_normalizatio... (BatchNormalizatio...	(None, 31, 31, 512)	2,048	conv2d_10[0][0]
leaky_re_lu_10 (LeakyReLU)	(None, 31, 31, 512)	0	batch_normalizat...
zero_padding2d_1 (ZeroPadding2D)	(None, 33, 33, 512)	0	leaky_re_lu_10[0]...
conv2d_11 (Conv2D)	(None, 30, 30, 1)	8,193	zero_padding2d_1...

Total params: 2,770,433 (10.57 MB)

Trainable params: 2,768,641 (10.56 MB)

Define Loss Functions and Optimizers

For a GAN, we need two loss functions: one for the Discriminator and one for the Generator.

- **Discriminator Loss:** Measures how well the discriminator distinguishes between real and fake images. It tries to maximize the accuracy of its predictions.
- **Generator Loss:** Measures how well the generator fools the discriminator, and also includes an L1 loss (Mean Absolute Error) to ensure the generated image is visually similar to the target image (sharp image). This L1 loss is crucial for image-to-image translation tasks.

We'll use Adam optimizers for both the generator and discriminator.

```
LAMBDA = 100

def discriminator_loss(disc_real_output, disc_generated_output):
    real_loss = tf.keras.losses.BinaryCrossentropy(from_logits=True)(tf.ones_like(disc_real_output), disc_real)
    generated_loss = tf.keras.losses.BinaryCrossentropy(from_logits=True)(tf.zeros_like(disc_generated_output), disc_generated_output)
    total_disc_loss = real_loss + generated_loss
    return total_disc_loss

def generator_loss(disc_generated_output, gen_output, target):
    gan_loss = tf.keras.losses.BinaryCrossentropy(from_logits=True)(tf.ones_like(disc_generated_output), disc_generated_output)
    # Mean absolute error
    l1_loss = tf.reduce_mean(tf.abs(target - gen_output))
    total_gen_loss = gan_loss + (LAMBDA * l1_loss)
    return total_gen_loss

generator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
```

Training Step Function

Now, let's define a single training step for our GAN. In each step:

1. The Generator takes a blurry image and produces a fake deblurred image.

2. The Discriminator then evaluates both the real deblurred image and the fake deblurred image.
3. The losses for both Generator and Discriminator are calculated.
4. The gradients are computed and applied to update the weights of both models.

```
@tf.function
def train_step(input_image, target_image):
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        gen_output = generator(input_image, training=True)

        disc_real_output = discriminator([input_image, target_image], training=True)
        disc_generated_output = discriminator([input_image, gen_output], training=True)

        gen_total_loss = generator_loss(disc_generated_output, gen_output, target_image)
        disc_loss = discriminator_loss(disc_real_output, disc_generated_output)

        generator_gradients = gen_tape.gradient(gen_total_loss, generator.trainable_variables)
        discriminator_gradients = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

        generator_optimizer.apply_gradients(zip(generator_gradients, generator.trainable_variables))
        discriminator_optimizer.apply_gradients(zip(discriminator_gradients, discriminator.trainable_variables))
```

✓ Full Training Loop

Finally, we'll put it all together into a training loop. This loop will iterate over the dataset for a specified number of epochs, calling the `train_step` function for each batch. During training, you would typically save checkpoints of your models and generate sample images to monitor progress. For this example, we'll focus on the basic loop structure.

```
import datetime
import numpy as np

log_dir="logs/fit/"
summary_writer = tf.summary.create_file_writer(
    log_dir + "" + datetime.datetime.now().strftime("%Y%m%d-%H%M%S"))
```

```
def fit(dataset, epochs):
    gen_epoch_losses = []
    disc_epoch_losses = []

    for epoch in range(epochs):
        start = datetime.datetime.now()

        total_gen_loss = 0
        total_disc_loss = 0
        num_batches = 0

        for n, (input_image, target_image) in enumerate(dataset):
            gen_loss, disc_loss = train_step(input_image, target_image)
            total_gen_loss += gen_loss
            total_disc_loss += disc_loss
            num_batches += 1

            if n % 500 == 0:
                print(f'. ', end='')
        print('')

        avg_gen_loss = total_gen_loss / num_batches
        avg_disc_loss = total_disc_loss / num_batches

        gen_epoch_losses.append(avg_gen_loss.numpy())
        disc_epoch_losses.append(avg_disc_loss.numpy())

        print(f'Epoch {epoch + 1} | Gen Loss: {avg_gen_loss:.4f} | Disc Loss: {avg_disc_loss:.4f} | Time: {dat

    return gen_epoch_losses, disc_epoch_losses

@tf.function
def train_step(input_image, target_image):
    with tf.GradientTape() as gen_tape, tf.GradientTape() as disc_tape:
        gen_output = generator(input_image, training=True)

        disc_real_output = discriminator([input_image, target_image], training=True)
        disc_generated_output = discriminator([input_image, gen_output], training=True)
```

```

gen_total_loss = generator_loss(disc_generated_output, gen_output, target_image)
disc_loss = discriminator_loss(disc_real_output, disc_generated_output)

generator_gradients = gen_tape.gradient(gen_total_loss, generator.trainable_variables)
discriminator_gradients = disc_tape.gradient(disc_loss, discriminator.trainable_variables)

generator_optimizer.apply_gradients(zip(generator_gradients, generator.trainable_variables))
discriminator_optimizer.apply_gradients(zip(discriminator_gradients, discriminator.trainable_variables))

return gen_total_loss, disc_loss

```

✓ Dataset Preparation for GAN Training

For GAN training, we need to load paired blurry and sharp images. We'll use the `RealBlur_J` dataset as an example. This dataset typically has a structure where blurry images are in one subdirectory and corresponding sharp images are in another.

We'll create a `tf.data.Dataset` pipeline to efficiently load, preprocess, and batch these image pairs.

```

# Define paths for blurry and sharp images within RealBlur_J
# Updated paths based on directory inspection
blurry_path = os.path.join(base_dir, 'CelebA', 'test', 'blur')
sharp_path = os.path.join(base_dir, 'CelebA', 'test', 'sharp')

# Get a list of all blurry and sharp image filenames
# It's crucial that the filenames match between blurry and sharp images
blurry_images = sorted([os.path.join(blurry_path, fname) for fname in os.listdir(blurry_path) if fname.lower().endswith('.png')])
sharp_images = sorted([os.path.join(sharp_path, fname) for fname in os.listdir(sharp_path) if fname.lower().endswith('.png')])

print(f"Found {len(blurry_images)} blurry images and {len(sharp_images)} sharp images in RealBlur_J.")

# Ensure the number of blurry and sharp images match
if len(blurry_images) != len(sharp_images):
    raise ValueError("Number of blurry and sharp images do not match. Please check your dataset.")

```

```
# Create a TensorFlow dataset from these paths
dataset = tf.data.Dataset.from_tensor_slices((blurry_images, sharp_images))
```

Found 1300 blurry images and 1300 sharp images in RealBlur_J.

```
print("Contents of CelebA:")
print(os.listdir(os.path.join(base_dir, 'CelebA')))
```

Contents of CelebA:
['train', 'test', 'validation']

```
print("Contents of CelebA/test:")
print(os.listdir(os.path.join(base_dir, 'CelebA', 'test')))
```

Contents of CelebA/test:
['blur', 'sharp']

```
def load_image(image_path):
    image = tf.io.read_file(image_path)
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.convert_image_dtype(image, tf.float32)
    image = tf.image.resize(image, [IMG_SIZE, IMG_SIZE], method=tf.image.ResizeMethod.NEAREST_NEIGHBOR)
    image = (image - 0.5) * 2 # Normalize to [-1, 1]
    return image

def load_paired_images_train(blurry_path, sharp_path):
    blurry_image = load_image(blurry_path)
    sharp_image = load_image(sharp_path)
    return blurry_image, sharp_image

# Map the loading function to the dataset
processed_dataset = dataset.map(load_paired_images_train,
                                num_parallel_calls=tf.data.AUTOTUNE)
```

```
# Shuffle, batch, and prefetch the dataset for optimal performance
BUFFER_SIZE = len(blurry_images) # Use total dataset size for full shuffling
```

```
train_dataset = processed_dataset.shuffle(BUFFER_SIZE).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)

print("Dataset pipeline created successfully. You can now use 'train_dataset' with the 'fit' function.")
```

Dataset pipeline created successfully. You can now use 'train_dataset' with the 'fit' function.

```
gen_losses, disc_losses = fit(train_dataset, epochs=1) # Use the custom fit function with the GAN-specific dat
```

```
.
Epoch 1 | Gen Loss: 12.3771 | Disc Loss: 1.0013 | Time: 0:44:07.223264 sec
```

```
import matplotlib.pyplot as plt

epochs_range = range(len(gen_losses))

plt.figure(figsize=(12, 6))
plt.plot(epochs_range, gen_losses, label='Generator Loss')
plt.plot(epochs_range, disc_losses, label='Discriminator Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Generator and Discriminator Losses Over Epochs')
plt.legend()
plt.grid(True)
plt.show()
```



✓ Deblur an uploaded image using the trained Generator

Now that we have a trained (even if for just one epoch) Generator, let's see it in action. We'll upload a blurry image, preprocess it, pass it through the Generator, and then visualize the original blurry input and the deblurred output.

```
from google.colab import files
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf # Make sure tf is imported for image processing

# Upload an image
uploaded = files.upload()

for file_name in uploaded.keys():
    print(f'Uploaded file: {file_name}')
    # Load and preprocess the image
    img_path = file_name
    img_raw = tf.io.read_file(img_path)
```

```
img = tf.image.decode_image(img_raw, channels=3)
img = tf.image.convert_image_dtype(img, tf.float32)
original_image_display = tf.image.resize(img, [IMG_SIZE, IMG_SIZE]) # For displaying the original

# Resize and normalize for the generator input (normalize to [-1, 1])
img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE], method=tf.image.ResizeMethod.NEAREST_NEIGHBOR)
img = (img - 0.5) * 2 # Normalize to [-1, 1]
img_array = tf.expand_dims(img, axis=0) # Add batch dimension

# Use the generator to deblur the image
deblurred_image = generator(img_array, training=False)
deblurred_image = (deblurred_image * 0.5) + 0.5 # Denormalize to [0, 1] for display
deblurred_image_display = tf.squeeze(deblurred_image, axis=0) # Remove batch dimension

# Display original blurry and deblurred images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(original_image_display)
plt.title('Original Blurry Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(deblurred_image_display)
plt.title('Deblurred Image (Generator Output)')
plt.axis('off')

plt.show()
```

Choose files Celebrity Blur Image.JPG

Celebrity Blur Image.JPG(image/jpeg) - 26757 bytes, last modified: 14/07/2026 - 100% done

Saving Celebrity Blur Image.JPG to Celebrity Blur Image.JPG

Uploaded file: Celebrity Blur Image.JPG

Original Blurry Image



Deblurred Image (Generator Output)



```
from google.colab import files
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf # Make sure tf is imported for image processing

# Upload an image
uploaded = files.upload()

for file_name in uploaded.keys():
    print(f'Uploaded file: {file_name}')
    # Load and preprocess the image
    img_path = file_name
    img_raw = tf.io.read_file(img_path)
    img = tf.image.decode_image(img_raw, channels=3)
    img = tf.image.convert_image_dtype(img, tf.float32)
    original_image_display = tf.image.resize(img, [IMG_SIZE, IMG_SIZE]) # For displaying the original
```

```
# Resize and normalize for the generator input (normalize to [-1, 1])
img = tf.image.resize(img, [IMG_SIZE, IMG_SIZE], method=tf.image.ResizeMethod.NEAREST_NEIGHBOR)
img = (img - 0.5) * 2 # Normalize to [-1, 1]
img_array = tf.expand_dims(img, axis=0) # Add batch dimension

# Use the generator to deblur the image

deblurred_image = generator(img_array, training=False)
deblurred_image = (deblurred_image * 0.5) + 0.5 # Denormalize to [0, 1] for display
deblurred_image_display = tf.squeeze(deblurred_image, axis=0) # Remove batch dimension

# Display original blurry and deblurred images
plt.figure(figsize=(10, 5))

plt.subplot(1, 2, 1)
plt.imshow(original_image_display)
plt.title('Original Blurry Image')
plt.axis('off')

plt.subplot(1, 2, 2)
plt.imshow(deblurred_image_display)
plt.title('Deblurred Image (Generator Output)')
plt.axis('off')

plt.show()
```

Choose files Sun Blur Image.JPG

Sun Blur Image.JPG(image/jpeg) - 15506 bytes, last modified: 14/07/2026 - 100% done

Saving Sun Blur Image.JPG to Sun Blur Image.JPG

Uploaded file: Sun Blur Image.JPG

Original Blurry Image



Deblurred Image (Generator Output)



✓ Evaluating Deblurring Quality: PSNR and SSIM

To quantitatively assess how well our model deblurred an image, we use metrics like PSNR and SSIM. These metrics require a **ground truth** sharp image to compare against the deblurred image generated by our model.

Below is an example of how you would calculate these metrics. Note that for your uploaded image, we do not have its true sharp counterpart. For demonstration purposes, we will use one of the sharp images from the `CelebA` dataset as a *placeholder* for `ground_truth_sharp_image` to illustrate the function calls.

```
import tensorflow as tf

# Assuming deblurred_image_display is available from the previous cell's execution
# (it represents the generator's output for the last uploaded image)

# --- Placeholder for Ground Truth Sharp Image ---
```

```
# In a real evaluation scenario, 'ground_truth_sharp_image' would be the actual
# sharp version corresponding to the blurry image you provide as input.
# For this demonstration, we'll load a sample sharp image from our dataset.

# Make sure 'sharp_images' list is still in memory from dataset preparation
if 'sharp_images' not in locals():
    print("Error: 'sharp_images' list not found. Please ensure dataset preparation cells have been run.")
else:
    # Load a sample sharp image from the dataset to act as a mock ground truth
    sample_sharp_image_path = sharp_images[0] # Using the first sharp image from the prepared list
    ground_truth_sharp_raw = tf.io.read_file(sample_sharp_image_path)
    ground_truth_sharp_image = tf.image.decode_jpeg(ground_truth_sharp_raw, channels=3)
    ground_truth_sharp_image = tf.image.convert_image_dtype(ground_truth_sharp_image, tf.float32)
    ground_truth_sharp_image = tf.image.resize(ground_truth_sharp_image, [IMG_SIZE, IMG_SIZE], method=tf.image.ResizeMethod.LANCZOS4)

    # Ensure both images are in the range [0, 1] for PSNR/SSIM calculation
    # deblurred_image_display is already in [0, 1] from previous cell
    # ground_truth_sharp_image is also in [0, 1] as loaded above

    print(f"--- Demonstrating PSNR and SSIM calculation ---")
    print(f>Note: This uses a sample sharp image from the dataset as a *mock* ground truth.")
    print(f"For meaningful results, you need the actual sharp version of the blurry image you are deblurring.")

    # Calculate PSNR
    psnr_value = tf.image.psnr(ground_truth_sharp_image, deblurred_image_display, max_val=1.0)

    # Calculate SSIM
    # The 'filter_size' and 'filter_sigma' can be adjusted for SSIM if needed.
    ssim_value = tf.image.ssim(ground_truth_sharp_image, deblurred_image_display, max_val=1.0)

    print(f"PSNR (Peak Signal-to-Noise Ratio): {psnr_value.numpy():.4f}")
    print(f"SSIM (Structural Similarity Index Measure): {ssim_value.numpy():.4f}")

    # Display the mock ground truth for context
    plt.figure(figsize=(6, 6))
    plt.imshow(ground_truth_sharp_image)
    plt.title('Mock Ground Truth Sharp Image (from CelebA dataset)')
    plt.axis('off')
```

```
--- Demonstrating PSNR and SSIM calculation ---
```

```
Note: This uses a sample sharp image from the dataset as a *mock* ground truth.
```

```
For meaningful results, you need the actual sharp version of the blurry image you are deblurring.
```

```
PSNR (Peak Signal-to-Noise Ratio): 5.8748
```

```
SSIM (Structural Similarity Index Measure): 0.3710
```

Could not connect to the reCAPTCHA service. Please check your internet connection and reload to get a reCAPTCHA challenge.